SECTION 01 · 시작 전 준비

us-stock 미국 주식 분석 시스템

Claude Code Plan Mode로 처음부터 구축 — Windows 10/11 완전 가이드

5개 파트 구성

Part 1 S&P 500 데이터 수집
Part 2 시장 체제 감지 (5-sensor)
Part 3 스마트머니 스크리닝
Part 4 AI 분석 리포트
Part 5 Next.js 대시보드 (13페이지)

자동화 목표

- 평일 07:00 Task Scheduler 자동 실행
- SQLite DB + JSON 파일 저장
- 전체 분석 소요: 5~15분
- 아침 5분으로 투자 결정 가능
- AI API 비용: 약 $0.20/일

총 예상 소요시간: 2~3시간 | **구독:** Claude Pro $20/월 (약 3만원) | **전제:** WSL2 필수 설치

SECTION 01

API 키 발급 가이드

아래 키를 메모장에 복사해두세요 — Phase 1 .env 파일에 입력합니다

키	필수	발급 URL	무료 한도
FRED_API_KEY	필수	발급 →	무제한
GOOGLE_API_KEY	AI용	발급 →	월 60회/분
OPENAI_API_KEY	선택	발급 →	유료
PERPLEXITY_API_KEY	선택	발급 →	유료
FINNHUB_API_KEY	선택	발급 →	분당 60회

최소 요건: FRED + Google Gemini 두 개만 있으면 핵심 기능 전부 작동. AI 키는 3개 중 1개 이상. 우선순위: Gemini > OpenAI > Perplexity

SECTION 01

Windows 환경 요건 확인

Windows 10 22H2+ / Python 3.13 / Node.js 20 LTS / WSL2 (Claude Code 필수)

POWERSHELL (관리자 권한) 복사

```powershell
# Python 3.13 설치
winget install Python.Python.3.13

# Node.js 20 LTS 설치
winget install OpenJS.NodeJS.LTS

# WSL2 설치 (Claude Code 필수!)
wsl --install

# 버전 확인 (재시작 후 PowerShell에서)
python --version   # Python 3.13.x
node --version     # v20.x.x
```

중요: Claude Code는 WSL2(Windows Subsystem for Linux)가 반드시 필요합니다. WSL2 설치 후 Ubuntu 터미널에서 모든 Claude Code 작업을 진행하세요.

팁: winget 대신 python.org에서 직접 다운로드도 가능합니다. 설치 시 "Add to PATH" 체크박스 반드시 선택!

SECTION 01

WSL2 Ubuntu 설정

Claude Code는 WSL2 Ubuntu 환경에서 실행 — Windows PowerShell 아님!

BASH (UBUNTU WSL2) 복사

```bash
# WSL2 Ubuntu 터미널 열기: 시작 → Ubuntu 검색

# Ubuntu 업데이트
sudo apt update && sudo apt upgrade -y

# Python 3.13 (Ubuntu PPA)
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt install -y python3.13 python3.13-venv

# 작업 디렉토리 생성
mkdir -p ~/projects && cd ~/projects
```

SECTION 01 완료 체크

- [ ] FRED API 키 발급 완료
- [ ] Google AI Studio API 키 발급 완료
- [ ] WSL2 Ubuntu 설치 완료 (PC 재시작)
- [ ] python3.13 --version 확인 (WSL2 내부)

다음: **SECTION 02 → Claude Code 설치**

Claude Code란?

터미널에서 동작하는 AI 코딩 에이전트 — 한국어로 대화하며 시스템 전체를 구축

Claude란?

Anthropic이 개발한 AI. 단순 답변을 넘어 함께 고민하는 지적 파트너 역할.

Opus 4.5: SWE-bench 80.8%

Sonnet 4.6: 일반 구현 (기본, 속도↑ 비용↓)

구독 플랜

Pro $20/월 — 이 프로젝트에 충분

Max 5x $100/월 — 헤비 유저

Max 20x $200/월 — 팀/전문가

API 사용량 기반 ($6/일 평균)

Windows 주의: Claude Code는 macOS에 최적화되어 있습니다. Windows에서는 반드시 WSL2 (Ubuntu) 환경에서 실행해야 합니다.

SECTION 02

WSL2 Ubuntu에서 Claude Code 설치

반드시 WSL2 Ubuntu 터미널에서 실행 — PowerShell 아님!

BASH (UBUNTU WSL2) 복사

```bash
# Node.js 20 설치 (WSL2 내부)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# bun 설치
curl -fsSL https://bun.sh/install | bash
source ~/.bashrc

# Claude Code 설치
bun install -g @anthropic-ai/claude-code

# 버전 확인
claude --version
```

주의: Windows PowerShell에서 설치하면 작동하지 않습니다. 반드시 WSL2 Ubuntu 터미널에서 실행하세요.

WSL2 안의 파일은 Windows 탐색기에서 \wsl$Ubuntuhome{사용자명}projects\us-stock 경로로 접근할 수 있습니다.

SECTION 02

로그인 & 초기 설정

WSL2 Ubuntu 터미널에서 최초 1회 실행

BASH (UBUNTU WSL2) 복사

```bash
# 프로젝트 디렉토리 생성
mkdir -p ~/projects/us-stock && cd ~/projects/us-stock

# Claude Code 실행 → 브라우저 자동 열림 → Anthropic 계정 로그인
claude

# 로그인 완료 후 종료: Ctrl+C 또는 /exit
```

참고: WSL2에서 실행 시 Windows 기본 브라우저가 자동으로 열립니다. claude.ai 계정으로 로그인 후 터미널로 복귀합니다.

SECTION 02

Plan Mode 사용법

코드를 수정하지 않고 설계만 — 전체 구축 계획을 검토 후 승인

단축키 / 명령	기능
Shift + Tab	**Plan Mode 진입** — 코드 수정 없이 설계만
/model opus	Opus로 전환 — 복잡한 설계·디버깅 시 사용
/model sonnet	Sonnet으로 전환 — 일반 구현 (기본)
/context	현재 토큰 사용량 확인 (비용 관리)
/compact	컨텍스트가 가득 찼을 때 요약 압축
/init	프로젝트에 맞는 CLAUDE.md 자동 생성
Esc	현재 작업 중단
Esc + Esc	이전 상태로 되돌리기 (Rewind)
y / n	파일 수정 권한 승인 / 거부

모델 전략: 대부분의 Phase는 Sonnet(기본)으로 충분합니다. 오류가 반복되거나 아키텍처 설계가 필요할 때만 /model opus로 전환하세요.

SECTION 02

Plan Mode로 구축 시작

windows_v2.md 전체를 복사해 Plan Mode에 붙여넣기

BASH (UBUNTU WSL2) 복사

```bash
# 1. 프로젝트 디렉토리로 이동
cd ~/projects/us-stock

# 2. Claude Code 실행
claude

# 3. Shift+Tab 눌러 Plan Mode 진입 → [PLAN MODE] 표시 확인

# 4. windows_v2.md 전체 내용 복사 → 붙여넣기 → Enter

# 5. Claude가 전체 구축 계획 작성 → 검토 후 승인 → Phase 0부터 자동 진행
```

SECTION 02 완료 체크

- [ ] WSL2 Ubuntu에서 bun 설치 완료
- [ ] claude --version 확인
- [ ] Anthropic 계정 로그인 완료
- [ ] Plan Mode (Shift+Tab) 진입 확인

다음: SECTION 03 → 프로젝트 구조 생성

SECTION 03 · 프로젝트 구조 생성

Phase 1: 디렉토리 구조 생성

WSL2 Ubuntu에서 전체 폴더 구조를 한 번에 생성

BASH (UBUNTU WSL2) 복사

```bash
cd ~/projects/us-stock

# 전체 디렉토리 구조 생성
mkdir -p src/{collectors,analyzers,ml/{pipeline,features/{equity,macro},va
mkdir -p scripts tests output/{picks,reports,models} data logs
mkdir -p .claude/agents/{ai,equity,frontend,macro,mlops,model,performance,
mkdir -p frontend/app frontend/src/{components,lib}

# __init__.py 생성
touch src/__init__.py src/collectors/__init__.py src/analyzers/__init__.py
touch src/ml/__init__.py src/ml/pipeline/__init__.py
touch src/ml/features/__init__.py src/ml/features/equity/__init__.py
touch src/ml/features/macro/__init__.py src/ml/validation/__init__.py
touch src/us_market/__init__.py src/db/__init__.py src/pipeline/__init__.p
```

WSL2(Linux 환경)이므로 Windows와 달리 슬래시(/) 경로 구분자를 사용합니다.

SECTION 03

가상환경 생성 & 패키지 설치

WSL2 Ubuntu에서 Python 3.13 가상환경 — Windows 방식과 활성화 명령이 다름!

BASH (UBUNTU WSL2) 복사

```bash
python3.13 -m venv .venv

# 활성화 (Linux 방식! .venvScriptsactivate 아님)
source .venv/bin/activate

pip install -r requirements.txt
```

REQUIREMENTS.TXT 복사

```
yfinance==1.2.0
fredapi==0.5.2
curl_cffi==0.13.0
pandas==3.0.2
numpy==2.4.4
scikit-learn==1.8.0
lightgbm==4.6.0
pyarrow==23.0.1
beautifulsoup4==4.14.3
requests==2.33.1
python-dotenv==1.2.2
matplotlib==3.10.8
seaborn==0.13.2
joblib==1.5.3
lxml==6.0.2
scipy==1.17.1
tqdm
optuna
```

주의: WSL2(Linux 환경)이므로 Windows 방식 .venvScriptsactivate가 아닌 source .venv/bin/activate를 사용합니다.

SECTION 03

.env 파일 설정

발급받은 API 키를 입력하세요

.ENV

복사

```
FRED_API_KEY=여기에_FRED_API_키_입력
GOOGLE_API_KEY=여기에_Gemini_API_키_입력
OPENAI_API_KEY=여기에_OpenAI_API_키_입력
PERPLEXITY_API_KEY=여기에_Perplexity_API_키_입력
FINNHUB_API_KEY=여기에_Finnhub_API_키_입력_선택사항
DATA_DB_PATH=output/data.db
```

보안: .env 파일은 절대 Git에 커밋하지 마세요. .gitignore에 .env가 포함되어 있는지 확인하세요.

SECTION 03

환경 검증

패키지 설치가 정상적으로 완료되었는지 확인

BASH (UBUNTU WSL2) 복사

```
source .venv/bin/activate
python -c "import yfinance, lightgbm, fredapi, curl_cffi, pandas; print('
```

SECTION 03 완료 체크

- [ ] 디렉토리 구조 생성 완료
- [ ] source .venv/bin/activate 정상 작동
- [ ] requirements.txt 설치 완료 (오류 없음)
- [ ] .env 파일 API 키 입력 완료

다음: SECTION 04 → Python 수집기 구축

src/collectors/ 구조

5개 파일 — S&P 500 종목, 주가, 매크로 데이터 수집

파일	역할
fetch_sp500_list.py	Wikipedia에서 S&P 500 종목 리스트 파싱 → CSV 저장
us_price_fetcher.py	USPriceFetcher — yfinance + curl_cffi Chrome 세션
macro_collector.py	MacroDataCollector — FRED API + VIX + Fear&Greed
data_fetcher.py	USStockDataFetcher — yfinance → Finnhub fallback
fetch_sp500_prices.py	503종목 배치 OHLCV 다운로드 (ThreadPoolExecutor)

핵심 패턴: curl_cffi Session(impersonate="chrome")으로 rate limit 회피. fetch_sp500_list.py는 __main__ 가드 없이 작성 (파이프라인에서 직접 import 사용)

SECTION 04

fetch_sp500_list.py

Wikipedia 파싱 → data/sp500_list.csv 저장

PYTHON 설계 명세 복사

```python
# Wikipedia에서 S&P 500 종목 리스트 파싱
# URL: https://en.wikipedia.org/wiki/List_of_S%26P_500_companies
# pandas.read_html(url)[0]으로 테이블 파싱
# Symbol 컬럼의 .을 -로 변환 (BRK.B → BRK-B)
# data/sp500_list.csv 저장 (Symbol, Security, GICS Sector)

def validate_sp500_list(df):
    # 행수 >= 500
    # GICS Sector 종류 >= 11
    # Symbol에 . 없음
    # 중복 없음

# 주의: if __name__ == "__main__" 가드 없이 작성!
```

SECTION 04

us_price_fetcher.py

curl_cffi Chrome 세션으로 yfinance rate limit 회피

PYTHON 설계 명세 복사

```python
from curl_cffi import requests as cffi_requests
import yfinance as yf

class USPriceFetcher:
    MAX_RETRIES = 3

    def __init__(self):
        self.session = cffi_requests.Session(impersonate="chrome")

    def fetch_ohlcv(self, symbol: str, period: str = "1y") -> pd.DataFrame
        # yf.download with session=self.session
        # exponential backoff retry (2**attempt 초 대기)
        # 빈 df 시 빈 DataFrame 반환 (절대 crash 없음)

    def fetch_batch(self, symbols: list, period: str = "1y") -> dict:
        # 여러 종목 수집. 종목 간 1초 sleep. {symbol: df} dict 반환
```

SECTION 04

macro_collector.py

FRED API + VIX + Fear&Greed 지수 수집

PYTHON 설계 명세 복사

```python
VIX_BOUNDARIES = {"risk_on": 16, "neutral": 22, "risk_off": 30}

class MacroDataCollector:
    def __init__(self):
        self.fred = Fred(api_key=os.getenv("FRED_API_KEY"))
        self.session = cffi_requests.Session(impersonate="chrome")

    def get_vix(self) -> dict:       # current_vix, ma20, trend
    def get_fred_data(self) -> dict: # DFF, T10Y2Y, BAMLH0A0HYM2
    def get_fear_greed(self) -> dict:# score, classification
    def get_macro_summary(self) -> dict
    def get_regime_from_vix(self, vix: float) -> str:
        # risk_on / neutral / risk_off / crisis
```

SECTION 04

수집기 검증

WSL2 Ubuntu 터미널에서 실행

BASH (UBUNTU WSL2) 복사

```bash
source .venv/bin/activate

python src/collectors/fetch_sp500_list.py
wc -l data/sp500_list.csv   # 500+ 줄 확인

python -c "
from src.collectors.us_price_fetcher import USPriceFetcher
p = USPriceFetcher()
df = p.fetch_ohlcv('AAPL')
print(f'AAPL 데이터: {len(df)}행, 컬럼: {list(df.columns[:5])}')
"
```

SECTION 04 완료 체크

- [ ] fetch_sp500_list.py 실행 → data/sp500_list.csv 생성
- [ ] USPriceFetcher AAPL 데이터 수집 성공
- [ ] MacroDataCollector FRED 연결 성공

다음: SECTION 05 → 분석기 구축

src/analyzers/ 구조

8개 파일 — 기술적 지표, 시장 체제, 스크리닝, AI 요약

파일	역할
technical_indicators.py	SMA/RSI/ATR/BB (순수 함수, df.copy() 반환)
sector_analyzer.py	SectorAnalyzer — 11 SPDR 섹터 ETF 분석
market_regime.py	MarketRegimeDetector — 5-sensor 가중 투표
market_gate.py	GO/CAUTION/STOP + 거래량-가격 다이버전스
smart_money_screener_v2.py	EnhancedSmartMoneyScreener — 6-factor 복합 점수
ai_response_parser.py	AI JSON 응답 파싱 + 검증
ai_summary_generator.py	Gemini/OpenAI/Perplexity 생성기
final_report_generator.py	FinalReportGenerator — Top 10 최종 리포트

SECTION 05

market_regime.py — 5-sensor 가중 투표

5개 센서의 가중 합산으로 시장 체제를 4단계로 판정

센서	가중치	소스
VIX	30%	^VIX
Trend	25%	SPY vs SMA50/200
Breadth	18%	RSP/SPY 상대강도
Credit	15%	HYG/IEF 비율
Yield Curve	12%	^TNX - ^IRX

체제	점수 범위
risk_on	< 0.75
neutral	0.75 ~ 1.5
risk_off	1.5 ~ 2.25
crisis	≥ 2.25

출력 파일: output/regime_result.json, output/regime_config.json

SECTION 05

smart_money_screener_v2.py

6개 팩터 가중 복합 점수 → 등급 A~F 판정

팩터	가중치	내용
Technical	25%	RSI/MACD/MA 신호
Fundamental	20%	PE/Growth/ROE
Analyst	15%	추천/목표가 괴리율
Relative Strength	15%	vs SPY 20일 비교
Volume	15%	SD 점수
13F Holdings	10%	기관 보유 (filing_date ≤ 어제)

등급 기준

A (80+) Strong Accumulation
B (70+) Moderate Accumulation
C (60+) Neutral
D (50+) Distribution
F (<50) Capitulation

Strategy/Setup

Trend / Swing / Reversal
Breakout / Pullback / Base

출력:

output/picks/smart_money_picks_YYYYMMDD.csv

SECTION 05

ai_summary_generator.py

Gemini → OpenAI → Perplexity 우선순위 fallback chain

PYTHON 설계 명세 복사

```python
class GeminiSummaryGenerator:
    MODEL = "gemini-2.5-flash-preview-04-17"
    # temperature=0.3
    # responseSchema=AI_OUTPUT_SCHEMA, responseMimeType="application/json"
    # 실패 시 _get_fallback_json(ticker) 반환 (절대 crash 없음)

class OpenAISummaryGenerator:
    MODEL = "gpt-4o-mini"

class PerplexitySummaryGenerator:
    MODEL = "sonar-pro"
    # timeout=90초, 실시간 웹 검색 포함

AI_OUTPUT_SCHEMA = {
    "thesis": str,          # 핵심 투자 논거 (50자 이상)
    "catalysts": list,      # 상승 촉매 3개
    "bear_cases": list,     # 하락 위험 3개 (필수)
    "recommendation": str,  # BUY/HOLD/SELL
    "confidence": int        # 0-100
}
```

SECTION 05

Verdict & Action 매핑

Regime + Gate + ML 예측을 종합해 최종 판정

Verdict 판정 로직

if regime in ("crisis","risk_off") or gate == "STOP" → STOP

elif regime == "risk_on" and gate == "GO" and ml_bullish → GO

else → CAUTION

Verdict	Grade A/B	Grade C	Grade D+
GO	BUY	WATCH	WATCH
CAUTION	SMALL BUY	WATCH	WATCH
STOP	HOLD	HOLD	HOLD

SECTION 05 완료 체크

- [ ] technical_indicators.py — add_all_indicators() 정상 작동
- [ ] market_regime.py — detect() 체제 판정 출력
- [ ] smart_money_screener_v2.py — 상위 20개 CSV 저장

다음: SECTION 06 → ML 파이프라인

SECTION 05

분석기 검증

WSL2 Ubuntu 터미널에서 실행

BASH (UBUNTU WSL2) 복사

```bash
source .venv/bin/activate

python -c "
from src.analyzers.market_regime import MarketRegimeDetector
d = MarketRegimeDetector()
result = d.detect()
print(f'체제: {result["regime"]}, 점수: {result["weighted_score"]:.2f}')
"

python -c "
from src.analyzers.market_gate import USMarketGate
gate = USMarketGate()
result = gate.analyze()
print(f'게이트: {result.gate}, 평균점수: {result.avg_score:.1f}')
"
```

src/ml/ 구조

피처 스토어 → 학습 → 추론 → Walk-Forward 검증

파일	역할
pipeline/feature_store.py	FeatureStore — parquet 저장/로드, 누출 검증
features/macro/build_macro_features.py	FRED 25개 피처 (Z-score, Momentum, Spread)
features/equity/build_equity_features.py	40+ 기술 지표 피처 (모멘텀/평균회귀/변동성)
pipeline/train.py	LightGBM rank_xendcg + Optuna HPO
pipeline/predict.py	최신 피처로 상위 20개 예측
validation/walk_forward.py	Walk-Forward + PBO + DSR

핵심 원칙: shift(1) 필수 (look-ahead bias 방지) | TimeSeriesSplit(n_splits=5) | SEED=42

SECTION 06

pipeline/train.py

시계열 교차검증 + Optuna HPO

PYTHON 설계 명세 복사

```python
SEED = 42
TARGET_COLUMNS = [
    "fwd_5d_return", "fwd_20d_return", "fwd_60d_return",
    "fwd_5d_rank", "fwd_20d_rank"
]

DEFAULT_LGBM_PARAMS = {
    "objective": "rank_xendcg",
    "metric": "ndcg",
    "num_leaves": 31,
    "learning_rate": 0.05,
    "seed": SEED,
}

def train_model(features_df, target_col="fwd_20d_rank"):
    # TimeSeriesSplit(n_splits=5) 시계열 교차검증
    # early_stopping(patience=50)
    # Optuna HPO 50 trials (일요일만)
    # 모델 저장: models/lgbm_{target}_{YYYYMMDD}.pkl
```

SECTION 06

validation/walk_forward.py

과적합 방지 + 통계적 신뢰성 검증

윈도우 설정

TRAIN_WINDOW = 730일 (2년)

VAL_WINDOW = 180일 (6개월)

TEST_WINDOW = 90일 (3개월)

EMBARGO = 20일 (누출 방지 갭)

검증 지표

PBO — Probability of Backtest Overfitting (López de Prado)

DSR — Deflated Sharpe Ratio (n_trials로 조정)

SECTION 06

build_equity_features.py

40+ 기술 지표 피처 — 카테고리별 분류

카테고리	개수	피처
모멘텀	6	ret_1m, ret_3m, ret_6m, ret_12m, ret_1m_minus_12m, ret_6m_skip1m
평균회귀	5	rsi_14, bb_pct_b, distance_52w_high/low, z_score_60d
변동성	6	vol_20d, vol_60d, atr_14, beta_60d, realized_skew, realized_kurt
거래량	4	vol_ratio_20_60, dollar_volume, amihud_illiq, turnover_rate
MA교차	4	price_ma20, price_ma50, ma20_slope, golden_cross

look-ahead bias 방지: cross-sectional rank 계산 시 pct=True | shift(1) 필수

SECTION 06

ML 파이프라인 검증

BASH (UBUNTU WSL2) 복사

```bash
source .venv/bin/activate

python src/ml/features/macro/build_macro_features.py
ls data/macro_features_*.parquet   # 파일 생성 확인

python src/ml/features/equity/build_equity_features.py
ls data/gbm_predictions.parquet
```

SECTION 06 완료 체크

- [ ] macro_features_*.parquet 생성 확인
- [ ] equity 피처 40+ 컬럼 확인
- [ ] train.py 실행 → models/ 디렉토리에 모델 저장

다음: SECTION 07 → DB + US Market

src/db/data_store.py

11개 테이블로 구성된 SQLite 데이터베이스

PYTHON 설계 명세 복사

```python
DB_PATH = os.getenv("DATA_DB_PATH", "output/data.db")

# 11개 테이블 스키마
# 시계열 (date PK): data_daily_reports, data_risk_alerts, data_prediction_h
# 스냅샷 (id=1 단일행): data_regime, data_market_gate, data_ai_summaries,
#   data_gbm_predictions, data_index_prediction, data_risk, data_performar

def get_db() -> sqlite3.Connection
def init_db(conn)           # 테이블 생성
def upsert_daily_report(conn, date, data: dict)
def upsert_regime(conn, data: dict)
def get_latest_report(conn) -> dict
```

SECTION 07

src/us_market/index_predictor.py

27개 피처로 SPY/QQQ 다음주 방향 예측

피처 그룹	개수
SPY (return/RSI/MACD/BB/volume/ATR)	7
VIX (level/change/ma20_diff)	3
QQQ (return_5d/spy_ratio)	2
시장폭 (advance_decline/new_high_low)	2
섹터 (XLK/XLY/XLU relative)	3
매크로 (yield_curve/credit_spread/fear_greed)	3
거래량 (spy_volume_surge/sector_trend/dark_pool)	3
모멘텀 (1m/3m/6m/roc_20)	4

출력: direction(bullish/bearish), probability(0-1), confidence(HIGH/MODERATE/LOW), key_drivers(상위 5개)

SECTION 07

DB + US Market 검증

BASH (UBUNTU WSL2) 복사

```bash
source .venv/bin/activate

python -c "
from src.db.data_store import get_db, init_db
conn = get_db()
init_db(conn)
print('DB 초기화 완료')
"

python -c "
from src.us_market.index_predictor import IndexPredictor
p = IndexPredictor('SPY')
result = p.predict_next_week()
print(f'SPY 예측: {result["direction"]}, 확률: {result["probability"]:.2f}')
"
```

SECTION 07 완료 체크

- [ ] output/data.db 생성 확인
- [ ] 11개 테이블 존재 확인
- [ ] IndexPredictor SPY/QQQ 예측 성공

다음: SECTION 08 → 실행 스크립트

SECTION 08 · 실행 스크립트

scripts/ 구조

5개 진입점 — 통합 분석, 전체 파이프라인, 자동화

스크립트	역할
run_integrated_analysis.py	Phase 0~3 통합 분석 (메인 실행 파일)
run_full_pipeline.py	9단계 전체 파이프라인 (데이터→DB→대시보드)
run_screening.py	S&P 500 스크리닝 단독 실행 (tqdm 진행 표시)
run_daily_scheduler.py	--status, --install-task, --time HH:MM
regen_dashboard_data.py	output/data.db → frontend/public/data/*.json

SECTION 08

run_integrated_analysis.py — 4단계 통합 분석

PYTHON 설계 명세 복사

```python
# argparse: --date YYYY-MM-DD (기본: 오늘)

# Phase 0: 데이터 최신성 체크
#   sp500_list.csv mtime > 7일이면 재수집
#   us_daily_prices.csv mtime > 1일이면 incremental 다운로드

# Phase 1: Verdict 판정
#   regime = MarketRegimeDetector().detect()
#   gate   = USMarketGate().analyze()
#   ml     = IndexPredictor("SPY").predict_next_week()

# Phase 2: 종목 선별 + Strategy/Setup 분류

# Phase 3: Action 매핑 (GO+A/B→BUY, CAUTION+A→SMALL BUY, STOP→HOLD)

# 결과: output/latest_report.json, SQLite upsert
```

SECTION 08

Task Scheduler 자동화 (Windows)

평일 07:00 자동 실행 — PowerShell (관리자 권한)에서 등록

POWERSHELL (관리자 권한) 복사

```
# WSL2 Ubuntu에서 Python 경로 확인 후 사용
$Action = New-ScheduledTaskAction -Execute "wsl" `
    -Argument "-d Ubuntu -e bash -c 'cd ~/projects/us-stock && source .ver

$Trigger = New-ScheduledTaskTrigger -Weekly `
    -DaysOfWeek Monday,Tuesday,Wednesday,Thursday,Friday `
    -At "07:00AM"

Register-ScheduledTask `
    -TaskName "US-Stock-Daily" `
    -Action $Action `
    -Trigger $Trigger `
    -RunLevel Highest `
    -Description "US Stock daily analysis pipeline"
```

팁: Task Scheduler는 PowerShell (관리자 권한)에서 실행하세요. WSL2 경로는 시스템마다 다를 수 있습니다.

SECTION 08

스크립트 검증

BASH (UBUNTU WSL2) 복사

```bash
source .venv/bin/activate
python scripts/run_integrated_analysis.py
ls -la output/latest_report.json   # 파일 존재 및 크기 > 0
```

POWERSHELL (관리자 권한) 복사

```powershell
Get-ScheduledTask -TaskName "US-Stock-Daily" | Select-Object TaskName, Sta
# State: Ready 이면 정상
```

SECTION 08 완료 체크

- [ ] run_integrated_analysis.py 실행 성공
- [ ] output/latest_report.json 파일 생성
- [ ] Task Scheduler "US-Stock-Daily" Ready 상태

다음: **SECTION 09 → Next.js 프론트엔드**

Next.js 15 대시보드

13개 페이지, API Routes, SQLite 연동, i18n (한국어/영어), 다크/라이트 테마

13개 페이지

/board — Reddit 스타일 게시판
/ — 홈 대시보드
/regime — 시장 체제 (5센서)
/top-picks — Top 10 종목
/ai — AI 분석 리포트
/forecast — SPY/QQQ 방향 예측
/ml — GBM ML 리더보드
/risk — 리스크 모니터
/performance — 성과 추적
/graph — 종목 관계도
/ai-builder — AI 에이전트 포털
/download — 자료 다운로드
/costs — API 비용 (항상 마지막)

핵심 의존성

next: ^15.1.6
react: ^19.0.0
better-sqlite3: ^12.9.0
recharts: ^3.8.1
react-force-graph-2d: ^1.29.1
tailwindcss: ^3.4.17
next-themes: ^0.4.6

SECTION 09

Node.js 설치 & 프론트엔드 빌드

WSL2 Ubuntu에서 npm 실행 — Windows Node.js 아님!

BASH (UBUNTU WSL2) 복사

```bash
cd ~/projects/us-stock/frontend
npm install
npm run build
npm run dev
# → http://localhost:3000 접속
```

참고: WSL2에서 실행된 localhost:3000은 Windows 브라우저에서 그대로 접속 가능합니다.

SECTION 09

frontend/app/api/data/ API Routes

better-sqlite3로 output/data.db를 쿼리해 JSON 반환

Route	데이터
reports/route.ts	일일 리포트 목록
regime/route.ts	시장 체제 데이터
market-gate/route.ts	마켓 게이트 GO/CAUTION/STOP
ai-summaries/route.ts	AI 종목 요약
index-prediction/route.ts	SPY/QQQ 예측
prediction-history/route.ts	예측 이력 (최대 100개)
gbm-predictions/route.ts	GBM 리더보드
risk/route.ts	리스크 모니터
performance/route.ts	성과 추적
graph/route.ts	종목 관계도
dates/route.ts	리포트 날짜 목록

SECTION 09

frontend/src/components/ 공유 컴포넌트

NAVIGATION.TSX (COSTS 항상 마지막)

복사

```
const NAV = [
  { href: "/board",       labelKey: "nav.board"       },
  { href: "/",            labelKey: "nav.overview"    },
  { href: "/regime",      labelKey: "nav.regime"      },
  { href: "/top-picks",   labelKey: "nav.topPicks"    },
  { href: "/ai",          labelKey: "nav.ai"          },
  { href: "/forecast",    labelKey: "nav.forecast"    },
  { href: "/ml",          labelKey: "nav.ml"          },
  { href: "/risk",        labelKey: "nav.risk"        },
  { href: "/performance", labelKey: "nav.performance" },
  { href: "/graph",       labelKey: "nav.graph"       },
  { href: "/ai-builder",  labelKey: "nav.aiBuilder"   },
  { href: "/download",    labelKey: "nav.download"    },
  { href: "/costs",       labelKey: "nav.costs"       }, // 항상 마지막
]
```

SECTION 09

프론트엔드 검증

BASH (UBUNTU WSL2) 복사

```bash
cd ~/projects/us-stock/frontend
npm install
npm run build   # TypeScript 에러 없음 확인

npm run dev &   # 백그라운드 실행
sleep 5 && curl -s -o /dev/null -w "%{http_code}" http://localhost:3000
# 200 이면 정상
```

SECTION 09 완료 체크

- [ ] npm run build 성공 (TypeScript 에러 없음)
- [ ] http://localhost:3000 접속 확인
- [ ] 13개 페이지 네비게이션 정상 동작
- [ ] 다크/라이트 테마 전환 확인

다음: SECTION 10 → 에이전트 & 최종 검증

.claude/agents/ 구조

10개 팀, 25개+ 에이전트 정의 파일

팀 구성

performance/ 5개 에이전트
equity/ 3개
model/ 4개
macro/ 3개
mlops/ 3개
system/ 3개
ai/ 3개
frontend/ 3개
research/ 4개
ingest/ 3개

모델 선택 기준

Lead 에이전트: **claude-opus-4-5**
일반 구현: **claude-sonnet-4-6**
단순 검증: **claude-haiku-4-5**

예시: gbm-trainer.md
model: claude-sonnet-4-6

SECTION 10

에이전트 파일 형식

.claude/agents/{팀}/{에이전트명}.md

.CLAUDE/AGENTS/MODEL/GBM-TRAINER.MD 복사

```
---
name: gbm-trainer
model: claude-sonnet-4-6
---

# GBM Trainer

## 역할
LightGBM 모델 학습과 하이퍼파라미터 최적화를 담당합니다.

## 담당 파일
- src/ml/pipeline/train.py
- src/ml/pipeline/feature_store.py

## 주요 기능
- TimeSeriesSplit 교차검증 실행
- Optuna HPO (일요일 50 trials)
- 모델 직렬화 및 메타데이터 저장

## 제약사항
- look-ahead bias 유발 코드 작성 금지
- 학습 데이터에 미래 수익률 직접 포함 금지
```

BASH (UBUNTU WSL2) 복사

```
find .claude/agents -name "*.md" | wc -l
# 25 이상이면 정상
```

SECTION 10

최종 검증 체크리스트

모든 Phase 완료 후 순서대로 확인

BASH (UBUNTU WSL2) 복사

```bash
source .venv/bin/activate

# 1. Python 패키지 확인
python -c "import yfinance, lightgbm, fredapi, curl_cffi, pandas; print('ㅍ

# 2. S&P 500 데이터
python src/collectors/fetch_sp500_list.py
wc -l data/sp500_list.csv   # 500+ 줄

# 3. 시장 체제 감지
python -c "from src.analyzers.market_regime import MarketRegimeDetector; p

# 4. 통합 분석
python scripts/run_integrated_analysis.py
ls -la output/latest_report.json

# 5. SQLite DB 확인
python -c "
import sqlite3; conn = sqlite3.connect('output/data.db')
tables = conn.execute("SELECT name FROM sqlite_master WHERE type='table'")
print(f'DB 테이블 수: {len(tables)}')
"

# 6. 프론트엔드 빌드
cd frontend && npm run build && echo "빌드 성공"
```

SECTION 10

구축 완료!

모든 항목이 정상이면 구축 완료입니다

최종 완료 체크

- [ ] Python 패키지 전체 설치 OK
- [ ] S&P 500 500개+ 종목 수집 완료
- [ ] 시장 체제 감지 정상 작동
- [ ] output/latest_report.json 생성 (크기 > 0)
- [ ] SQLite DB 11개 테이블 확인
- [ ] Next.js 빌드 성공 (TypeScript 오류 없음)
- [ ] http://localhost:3000 접속 정상
- [ ] Task Scheduler "US-Stock-Daily" Ready
- [ ] .claude/agents/ 25개+ 파일 생성

완료 후 일상 루틴

평일 07:00: Task Scheduler 자동 실행

아침: http://localhost:3000 접속

Overview → Regime → Top Picks 순서로 확인

5분 만에 투자 결정

문제 발생 시

WSL2에서 claude 실행

Plan Mode (Shift+Tab) 진입

오류 메시지 붙여넣기

Claude가 자동 수정 제안